An Explicit Infinite Model Refuting Ulrich's `u4` as a Single Axiom for Positive Implication

Byungwoong Yoo
Independent Researcher

Corrected strengthened release v2
Lean 4.33.0, leanchecker, nanoda, axiom-audit, and symbolic-check suite
Proof-source commit `fec164ed763794330d57f76ddd1b0c390f22db09`
All-green GitHub Actions run 32384585278
Verification snapshot: 20 August 2026 UTC (21 August 2026 KST)
Reserved corrected-v2 DOI: `10.5281/zenodo.22031656`
Corrected v2 publication: 21 August 2026 KST

Abstract

Fitelson and Peltier (2026) eliminated three of Ulrich's four remaining 15-symbol candidates for a single axiom of positive implicational logic, leaving

$$((x \to y) \to z) \to ((y \to (z \to u)) \to (y \to u))$$

as an open case. We construct an explicit infinite model satisfying every substitution instance of this formula and closed under modus ponens, while falsifying reflexivity $q \to q$. The model is the union of the substitution instances of a recursively defined family of formula schemes. The decisive countermodel property, uniform-substitution closure, the induced proof system, and an internal separation from the standard positive-implication basis are formalized in Lean 4.33.0. At the proof-source commit identified above, the Lean kernel, `leanchecker`, the independently implemented Rust checker `nanoda`, a compiled-environment axiom audit, and a separate Python symbolic checker all succeeded. These are machine-verification results within the project; external specialist and journal review remain pending.

1 Background

Carew Meredith found single axioms of length 17 for positive implication. Dolph Ulrich reduced the possibility of a shorter axiom to four formulas of length 15. Fitelson and Peltier used saturation-based theorem proving to eliminate the first three and reported that the fourth candidate,

$$\Phi = ((x \to y) \to z) \to ((y \to (z \to u)) \to (y \to u)), \tag{1}$$

remained open. They further established that the shortest single axioms have length 17 if and only if Φ is not a single axiom.

The proof below is semantic. Its domain is the free algebra of finite implicational formulas. The theorem predicate is a particular substitution- closed set that is also closed under modus ponens.

2 Recursive schemes

Write

$$C(A, B, R) := (A \to (B \to R)) \to (A \to R),$$

so that

$$\Phi = ((x \to y) \to z) \to C(y, z, u).$$

Choose pairwise distinct variables

$$a, b, c, r_2, r_3, \ldots$$

and define

$$T_1 = \Phi,$$

$$A_2 = a, \qquad B_2 = C(b, a, c), \qquad T_2 = C(A_2, B_2, r_2).$$

For $n \geq 2$, recursively set

$$A_{n+1} = B_n \to r_n, \qquad B_{n+1} = A_n \to r_n,$$

$$T_{n+1} = C(A_{n+1}, B_{n+1}, r_{n+1}).$$

For $n \geq 2$, put

$$D_n = A_n \to (B_n \to r_n),$$

so that

$$T_n = D_n \to (A_n \to r_n).$$

For a formula scheme S, let $\mathrm{Inst}(S)$ denote the set of all finite substitution instances of S.

3 The interlocking property

For finite formulas P, Q, let $E(P, Q)$ mean that no substitution σ and finite formula W satisfy

$$\sigma(P) = \sigma(Q) \to W.$$

Lemma 1 (Interlocking). *For every $n \geq 2$,*

$$E(A_n, B_n) \qquad \textit{and} \qquad E(B_n, A_n).$$

Proof. At $n = 2$,

$$A_2 = a, \qquad B_2 = ((b \to (a \to c)) \to (b \to c)).$$

If $\sigma(a) = \sigma(B_2) \to W$, then the finite formula $\sigma(a)$ occurs as a proper subformula of its own right-hand side. This is impossible. Conversely, if $\sigma(B_2) = \sigma(a) \to W$, injectivity of implication forces

$$\sigma(b \to (a \to c)) = \sigma(a),$$

which again makes $\sigma(a)$ a proper subformula of itself.

Assume both assertions at n. Since

$$A_{n+1} = B_n \to r_n, \qquad B_{n+1} = A_n \to r_n,$$

an equality

$$\sigma(A_{n+1}) = \sigma(B_{n+1}) \to W$$

would imply

$$\sigma(B_n) = \sigma(A_n) \to \sigma(r_n),$$

contradicting $E(B_n, A_n)$. The converse direction is symmetric. $\square$

4 The major-premise obstruction

Lemma 2. *For every $m \geq 2$ and $n \geq 1$, the schemes D_m and T_n have no common substitution instance.*

Proof. First let $n \geq 2$, and suppose

$$\sigma(D_m) = \tau(T_n).$$

Expanding both sides and using injectivity of implication gives

$$\sigma(A_m) = \tau(A_n) \to (\tau(B_n) \to \tau(r_n))$$

and

$$\sigma(B_m) = \tau(A_n).$$

Thus

$$\sigma(A_m) = \sigma(B_m) \to (\tau(B_n) \to \tau(r_n)),$$

contrary to Lemma 1.

Now let $n = 1$. If $m \geq 3$, then

$$A_m = B_{m-1} \to r_{m-1}, \qquad B_m = A_{m-1} \to r_{m-1}.$$

Matching D_m with T_1 forces both

$$\sigma(r_{m-1}) = \tau(z) \quad \text{and} \quad \sigma(r_{m-1}) = \tau(z \to u),$$

hence $\tau(z) = \tau(z) \to \tau(u)$, impossible for a finite formula.

For $m = 2$, a common instance of

$$D_2 = a \to (C(b, a, c) \to r_2)$$

and T_1 would imply

$$\sigma(a) = \tau((x \to y) \to z)$$

and

$$\sigma(C(b, a, c)) = \tau(y \to (z \to u)).$$

The second equality yields

$$\tau(y) = \sigma(b \to (a \to c)), \quad \tau(z) = \sigma(b), \quad \tau(u) = \sigma(c).$$

Substitution into the first equality makes $\sigma(a)$ contain itself as a proper subformula, a contradiction. □

5 The infinite model

Define

$$\mathcal{P} = \bigcup_{n \geq 1} \mathrm{Inst}(T_n).$$

Interpret the unary theorem predicate p on the free algebra of finite implicational formulas by

$$p(F) \quad \Longleftrightarrow \quad F \in \mathcal{P}.$$

Every instance of $\Phi = T_1$ belongs to $\mathcal{P}$. It remains to prove closure under modus ponens.

Lemma 3. *If $X \in \mathcal{P}$ and $X \to Y \in \mathcal{P}$, then $Y \in \mathcal{P}$.*

Proof. Choose $n, m \geq 1$ and substitutions α, β with

$$X = \alpha(T_n), \qquad X \to Y = \beta(T_m).$$

If $m \geq 2$, the antecedent of $\beta(T_m)$ is $\beta(D_m)$. Then X would be a common instance of D_m and T_n, contrary to Lemma 2. Hence $m = 1$, so

$$X = \beta((x \to y) \to z), \qquad Y = \beta(C(y, z, u)).$$

If $n = 1$, rename the variables of the minor copy of T_1 as x', y', z', u'. Comparison of

$$\alpha(T_1) = \beta((x \to y) \to z)$$

gives

$$\beta(y) = \alpha(z'), \qquad \beta(z) = \alpha(C(y', z', u')).$$

Therefore

$$Y = C\big(\alpha(z'), \alpha(C(y', z', u')), \beta(u)\big),$$

an instance of $T_2 = C(a, C(b, a, c), r_2)$.

If $n \geq 2$, comparison of

$$\alpha(T_n) = \beta((x \to y) \to z)$$

gives

$$\beta(y) = \alpha(B_n \to r_n), \qquad \beta(z) = \alpha(A_n \to r_n).$$

Hence

$$Y = C\big(\alpha(B_n \to r_n), \alpha(A_n \to r_n), \beta(u)\big),$$

which is an instance of T_{n+1}. Thus $Y \in \mathcal{P}$. $\square$

6 Failure of reflexivity

Lemma 4. *No formula $Q \to Q$ belongs to $\mathcal{P}$.*

Proof. For $n \geq 2$, an instance of

$$T_n = (A_n \to (B_n \to r_n)) \to (A_n \to r_n)$$

could equal $Q \to Q$ only if

$$\sigma(B_n \to r_n) = \sigma(r_n).$$

This would make the finite formula $\sigma(r_n)$ contain itself as a proper subformula.

For $n = 1$, equality of the antecedent and consequent of an instance of T_1 forces

$$\sigma(y) = \sigma(z \to u), \qquad \sigma(z) = \sigma(y \to u),$$

a contradiction by strict size increase in both directions. $\square$

Theorem 5. *Ulrich's formula* (1) *is not a single axiom for positive implicational logic under substitution and modus ponens.*

Proof. The interpretation above satisfies every substitution instance of Φ, and Lemma 3 proves that it satisfies modus ponens. Lemma 4 shows that it falsifies reflexivity $Q \to Q$, which is a theorem of positive implication. Therefore reflexivity is not derivable from Φ, so Φ is not a single axiom. $\square$

Corollary 6. *Combining Theorem 5 with the elimination and classification results established by Fitelson and Peltier, the minimum length of a single axiom for positive implication is 17 symbols.*

7 Formal certificate and verified release

The evidence bundle includes `lean/U4Formal.lean`. It defines the free algebra of finite implicational formulas, the exact formula Φ, and an explicit predicate P. The Lean kernel-checked theorem is

```
theorem U4Formal.u4_countermodel_exists :
    Exists fun pred : Fml -> Prop =>
      (forall x y z u, pred (Phi x y z u)) /\
      (forall X Y, pred X -> pred (I X Y) -> pred Y) /\
      (forall q, Not (pred (I q q)))
```

The strengthened file additionally formalizes arbitrary uniform substitution, proves that every syntactic `u4` derivation is sound in the model, and derives reflexivity from the standard K and S schemes. Its final theorem exhibits a positive-implication theorem that is not derivable from `u4`:

```
theorem U4Formal.u4_not_axiomatizes_positive_implication :
    Exists fun f : Fml => PositiveDerivable f /\ Not (U4Derivable f)
```

The source scan forbids `sorry`, `admit`, custom `axiom`, and `unsafe`; the compiled-environment audit is the check of the declared axiom-dependency boundary. The CI artifact records the exact Git commit, all checker outcomes, direct compiler output, source files, and SHA-256 hashes. Run 32384585278 passed every gate at commit `fec164ed763794330d57f76ddd1b0c390f22db09`. The same run reports that `nanoda` checked 96,602 declarations with no typechecker errors; its one reported diagnostic concerned pretty-printing axioms and not type checking.

8 Executable cross-checks

The release workflow performs the following mutually reinforcing checks.

Check	Range	Result
Lean build and direct compilation	Lean 4.33.0	passed
Lean environment recheck	`leanchecker`	passed
Separately implemented Rust type check	`nanoda`, no `sorryAx` use	passed
Compiled axiom audit	standard allowlist only	passed
Python recurrence/interlocking/major obstruction	$T_1, \dots, T_{100}$	passed
Python exhaustive binary-tree enumeration	through 12 leaves	passed

At 12 leaves there are $C_{11} = 58{,}786$ binary application-tree shapes. Exactly one was typable: the right comb. These checks support, but do not replace, the all-n proof above.

9 Verification status and claim boundary

As of 21 August 2026, the official 2026 source cited below reported `u4` as open. This release makes no claim that all later, unpublished, or non-indexed work has been identified.

The public identifiers are:

Corrected v2 (reserved)	`10.5281/zenodo.22031656`
All-version concept	`10.5281/zenodo.21987698`
Historical v1	`10.5281/zenodo.21987699`

The v1 record contains a Lean 4.30.0 kernel check of the core countermodel. This corrected strengthened release adds explicit substitution and derivability, an internal K/S separation theorem, and multiple independently implemented compiled checks. The mathematical exposition, the connection to the prior reduction, external specialist review, journal peer review, and worldwide priority remain separate questions. The defensible description is:

> Lean-checked explicit infinite countermodel, supported by project-run checks using separately implemented environment and type checkers, showing that Ulrich's `u4` is not a single axiom. No independent third-party specialist or journal review has yet occurred, and comparative priority review remains pending.

AI-use disclosure. OpenAI GPT-5-class systems, including Codex, substantially assisted with proof exploration, adversarial checking, program drafting, literature-search assistance, and manuscript preparation. The human author, Byungwoong Yoo, finalized the mathematical argument and the submitted package, reviewed the claim and evidence, and assumes full responsibility for the accuracy and final wording. Verification and replay details are documented separately in the supplied logs and checkers.

References

[1] B. Fitelson and N. Peltier, "Applying Saturation-Based Theorem Proving to Open Problems in Positive Implicational Logic," *Journal of Automated Reasoning*, vol. 70, article 5, 2026. doi:10.1007/s10817-026-09752-1.

[2] J. R. Hindley and D. Meredith, "Principal Type-Schemes and Condensed Detachment," *Journal of Symbolic Logic*, vol. 55, no. 1, pp. 90–105, 1990. doi:10.2307/2274956.

[3] J. A. Kalman, "Condensed Detachment as a Rule of Inference," *Studia Logica*, vol. 42, no. 4, pp. 443–451, 1983. doi:10.1007/BF01371632.

[4] D. Ulrich, "New Single Axioms for Positive Implication," *Bulletin of the Section of Logic*, vol. 28, pp. 39–42, 1999.

[5] D. Ulrich, "A Legacy Recalled and a Tradition Continued," *Journal of Automated Reasoning*, vol. 27, no. 2, pp. 97–122, 2001.

[6] C. A. Meredith, "A Single Axiom of Positive Logic," *Journal of Computing Systems*, vol. 1, no. 3, pp. 169–170, 1953.